ALSHUMOOKH GROUP

Settlement Gateway v2.0 — Technical Integration Division

TECHNICAL INTEGRATION ROADMAP & API SPECIFICATION

HIPERCAPITAL FINANCE ↔ ALSHUMOOKH SETTLEMENT GATEWAY v2.0

VERSION 5.0 — COMPLETE TECHNICAL SPECIFICATION

Prepared by:	ALSHUMOOKH GROUP — Technical Operations
Document Type:	External Integration Specification (Full API Reference)
Classification:	CONFIDENTIAL — For HIPERCAPITAL FINANCE Only
Date:	08 June 2026
Reference:	ASIG-HIPC-TECH-2026-v5
Gateway:	Alshumookh Settlement Gateway v2.0 (FastAPI / Python 3.11)

TABLE OF CONTENTS

1. EXECUTIVE SUMMARY & SYSTEM OVERVIEW

ALSHUMOOKH GROUP operates the Alshumookh Settlement Gateway v2.0 (ASIG), a production-grade FastAPI blockchain settlement platform. This document gives HIPERCAPITAL FINANCE every technical detail required to integrate with ASIG — all endpoints, field aliases, status codes, error codes, and request examples are taken directly from the live codebase.

Component	ASIG (Production)	HIPERCAPITAL Requirement
Gateway	Alshumookh Settlement Gateway v2.0	REST API integration over HTTPS
Framework	FastAPI · Python 3.11+	Standard JSON/HTTPS client
Accepted Assets	USDT (ERC-20 / TRC-20) · USDC · M1 Token	HCEURO must bridge to USDT first
Supported Networks	Ethereum · Base · Tron	HIPCF Chain NOT natively supported
Blockchain Verify	Alchemy Webhooks + EVM RPC	Public-chain tx_hash is mandatory
Auth Methods	API Key · OAuth2 · HMAC · JWS · JWE · mTLS	Minimum: API Key + HMAC-SHA256
Rate Limit	240 req/60s global · 200 req/60s per key	Must implement request queuing
Confirmations	6+ blocks (ETH/Base) · 20 blocks (Tron)	Wait before marking settled

✗ CRITICAL BLOCKER:

ASIG does NOT accept: asset = TOKENIZED_USD, USDX, or HCEURO, nor settlement_type = PRIVATE_INTERNAL_LEDGER. HIPERCAPITAL must bridge HCEURO to USDT on Ethereum or Base before any payload submission.

2. SECURITY ARCHITECTURE & REQUEST PIPELINE

■ PIPELINE:

Every inbound request passes through these stages in order:

1. IP Allowlist Check — caller IP must be registered for the API client
2. API Key / OAuth2 Authentication — X-API-Key or Bearer token validated
3. Idempotency-Key Check — duplicate request detection and replay
4. mTLS Certificate Verification — if client requires mutual TLS
5. JWE Decryption — if body is JWE-encrypted (Content-Type: application/jose)
6. Timestamp Validation — X-Timestamp must be within 5 minutes (UTC)
7. JWS Signature Verification — if client requires signed payloads
8. HMAC-SHA256 Validation — X-HMAC-Signature verified against shared secret
9. JSON Normalization — field alias resolution to canonical names
10. Database Persist — payload stored with initial verification_status
11. Alchemy Blockchain Verification — tx_hash verified on-chain via webhook
12. Confirmation Threshold — 6+ blocks (ETH/Base) or 20 blocks (Tron)
13. Reconciliation — admin confirms and marks RECONCILED

2.1 Security Layers Summary

Security Layer	Header / Mechanism	Required?	Notes
IP Allowlist	Server-side IP check	**Required**	Egress IPs registered in ASIG dashboard
API Key	`X-API-Key: <key>`	**Required**	Generated from ALSHUMOOKH admin panel
Idempotency	`Idempotency-Key: <uuid4>`	**Required**	Unique UUID per request — deduplication

Security Layer	Header / Mechanism	Required?	Notes
OAuth2 Bearer	`Authorization: Bearer <token>`	**Optional**	Required if client config sets require_oauth=true
HMAC-SHA256	`X-HMAC-Signature: <hex>`	**Recommended**	Signs method + url + timestamp + body hash
Timestamp	`X-Timestamp: <unix>`	**With HMAC**	Max 5-minute drift from server UTC time
JWS Signing	`X-JWS-Signature: <token>`	Optional	JSON Web Signature over full request body
JWE Encrypt	`Content-Type: application/jose`	Optional	Full body as JWE compact serialization
mTLS	`X-Client-Cert-Fingerprint: <SHA256>`	Optional	Client cert SHA-256 fingerprint validation

3. AUTHENTICATION METHODS — COMPLETE SPECIFICATION

3.1 API Key — Minimum Required for All Requests

Every request to ASIG must include the X-API-Key header. Keys are generated from the ALSHUMOOKH admin dashboard. An inactive or missing key returns HTTP 401 with error_code: invalid_api_key.

Required headers:

```
X-API-Key: asig_live_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
```

3.2 OAuth2 Client Credentials Flow

POST to /api/v1/oauth/token with client_id and client_secret. The returned Bearer token expires in 3600 seconds. Required when client config sets require_oauth = true.

Token request:

```
POST /api/v1/oauth/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id=YOUR_CLIENT_ID
&client_secret=YOUR_CLIENT_SECRET

Response:
{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
"token_type": "bearer",
"expires_in": 3600 }
```

Use token in subsequent requests:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

3.3 HMAC-SHA256 Request Signing — Recommended

The HMAC secret is assigned per API client. The signature covers: HTTP method + URL + Unix timestamp + SHA-256 hash of the request body. Maximum allowed clock drift is 5 minutes (UTC).

Python construction:

```
import hmac, hashlib, time, json

body = json.dumps(payload) # serialized JSON body
timestamp = str(int(time.time())) # Unix timestamp (UTC)
body_hash = hashlib.sha256(
body.encode()).hexdigest()
message = f'POST\n/api/v1/payloads/ingest
\n{timestamp}\n{body_hash}'
signature = hmac.new(
secret.encode(),
message.encode(),
hashlib.sha256).hexdigest()

# Add to request headers:
X-HMAC-Signature: <signature_hex>
X-Timestamp: <unix_timestamp>
```

4. MAIN INGEST ENDPOINT — POST /api/v1/payloads/ingest

Property	Value
Endpoint	`POST /api/v1/payloads/ingest`
Content-Type	application/json (or application/jose for JWE payload)
Required Headers	`X-API-Key + Idempotency-Key: <uuid4>`
Optional Headers	X-HMAC-Signature · X-Timestamp · Authorization: Bearer · X-JWS-Signature
Rate Limit	200 requests per 60 seconds per API key | 240 per 60 seconds global
Success — with tx_hash	HTTP 200 · status: "payload_received" · verification_status: "ALCHEMY_PENDING"
Success — no tx_hash	HTTP 200 · status: "payload_received" · verification_status: "AWAITING_TX_HASH"
Parse Failed	HTTP 200 · status: "payload_received_unparsed" · verification_status: "RECEIVED"
Idempotency Replay	HTTP 200 · status: "payload_received" · (original response replayed)
Auth Failure	HTTP 401 / 403 · JSON body with error_code — see Section 7
Validation Failure	HTTP 400 · missing required field or malformed request body

4.1 Complete cURL Request Example

```
curl -X POST https://api.alshumookh.com/api/v1/payloads/ingest \
-H 'Content-Type: application/json' \
-H 'X-API-Key: asig_live_xxxxxxxxxxxxxxxxxxxx' \
-H 'Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000' \
-H 'X-HMAC-Signature: 3a7f9c2e4b8d1f...' \
-H 'X-Timestamp: 1749461234' \
-d '{
"transaction_reference": "HIPC-2026-001",
"tx_hash": "0xabc123...def456",
"sender_wallet": "0xYOUR_SOURCE_WALLET_ADDRESS",
"receiver_wallet": "0xBD682cfD8382a90adfDd6745780D3D7959c4d939",
"amount": "50000.00",
"asset": "USDT",
"network": "ethereum",
"token_contract": "0xdAC17F958D2ee523a2206206994597C13D831ec7",
"settlement_type": "wallet_to_wallet",
"callback_url": "https://your-server.com/webhook/asig"
}'
```

4.2 Response Reference Table

Scenario	HTTP	Key Response Fields
Normal — with tx_hash	200	`status: "payload_received" · verification_status: "ALCHEMY_PENDING" · payload_id: <uuid>`
No tx_hash provided	200	`status: "payload_received" · verification_status: "AWAITING_TX_HASH"`
JSON parse failed	200	`status: "payload_received_unparsed" · verification_status: "RECEIVED"`
Idempotency replay	200	`status: "payload_received" · parsed: true/false (original response replayed)`
Invalid API key	401	`{ "error": "invalid_api_key", "message": "API key is invalid or inactive" }`
Missing Idempotency-Key	400	`{ "error": "missing_idempotency_key", "message": "Idempotency-Key is required" }`
IP not allowed	403	`{ "error": "ip_not_allowed", "message": "Client IP is not allowed" }`

5. COMPLETE JSON SCHEMA & ALL FIELD ALIASES

ASIG's normalization engine resolves incoming field names using alias maps. HIPERCAPITAL may use their own naming — the engine matches aliases at the top level or one level deep inside nested objects. Red = required, Amber = recommended.

Canonical Field	Accepted Aliases (any of these maps to the canonical name)	Type	Level
`transaction_reference`	transaction_id · txRef · reference · ref · transactionCode · referenceId · payment_reference	string	**Recommended**
`tx_hash`	transaction_hash · hash · blockchain_hash · txid · txHash · transactionHash	string	**Conditional**
`sender_wallet`	from_wallet · from · source_wallet · fromAddress · from_address · senderAddress · origin_wallet	address	**Required**
`receiver_wallet`	to_wallet · to · destination_wallet · toAddress · to_address · recipientAddress · beneficiary_wallet	address	**Required**
`amount`	value · token_amount · transfer_amount · settlement_amount · tokenAmount · transferAmount	decimal str	**Required**
`asset`	currency · token · symbol · crypto_currency · tokenSymbol · cryptoCurrency	string	**Required**
`network`	chain · blockchain · protocol · chainName · networkName	string	**Required**
`token_contract`	contract_address · contract · token_address · contractAddress · tokenContract	address	**Recommended**
`timestamp`	created_at · time · tx_time · txTime · transaction_time	ISO8601	Optional
`sender`	sender_name · from_name · senderName · fromName · payer	string	Optional
`receiver`	receiver_name · to_name · beneficiary · receiverName · toName · payee	string	Optional
`settlement_type`	settlement · type · settlementType · paymentType · payment_type	string	Optional
`authorization_code`	auth_code · authorization · authCode · authorizationCode	string	Optional
`callback_url`	callback · webhook_url · notify_url · callbackUrl · webhookUrl · notifyUrl	URL	Optional
`signature`	sig · x_signature · xSignature · payloadSignature	hex	Optional
`payload_hash`	hash_payload · content_hash · payloadHash · contentHash	SHA-256	Optional

5.1 Full JSON Payload Example — All Fields

```
{
"transaction_reference": "HIPC-2026-001",
"tx_hash": "0xabc123def456...",
"sender_wallet": "0xYOUR_SOURCE_WALLET_ADDRESS",
"receiver_wallet": "0xBD682cfD8382a90adfDd6745780D3D7959c4d939",
"amount": "50000.00",
"asset": "USDT",
"network": "ethereum",
"token_contract": "0xdAC17F958D2ee523a2206206994597C13D831ec7",
"settlement_type": "wallet_to_wallet",
"timestamp": "2026-06-09T10:00:00Z",
"sender": "HIPERCAPITAL FINANCE",
"receiver": "ALSHUMOOKH GROUP",
"authorization_code": "HIPC-AUTH-2026",
"callback_url": "https://your-server.com/webhook/asig"
}
```

5.2 HIPERCAPITAL Alternative Field Names — Auto-Resolved by ASIG

```
{
"txRef": "HIPC-2026-001", // -> transaction_reference
"txHash": "0xabc123...", // -> tx_hash
"fromAddress": "0xYOUR_WALLET", // -> sender_wallet
"toAddress": "0xBD682c...", // -> receiver_wallet
"tokenAmount": "50000.00", // -> amount
"tokenSymbol": "USDT", // -> asset
"chainName": "ethereum", // -> network
"contractAddress": "0xdAC17F...", // -> token_contract
"webhookUrl": "https://...", // -> callback_url
"authCode": "HIPC-AUTH-2026" // -> authorization_code
}
```

6. ALL STATUS CODES (Payload · Transfer · Order)

6.1 PayloadVerificationStatus — Inbound Settlement Flow

Status	Meaning	Trigger	Required Action
RECEIVED	Stored but parsing failed or unrecognized structure	Ingest — parse fail	Check JSON format · contact ASIG ops
PARSED	Fields normalized and extracted successfully	Ingest — no tx_hash	Submit tx_hash when transaction is broadcast
AWAITING_TX_HASH	Parsed but no blockchain tx_hash provided	Ingest without tx_hash	PATCH payload with tx_hash once broadcast
ALCHEMY_PENDING	tx_hash submitted — awaiting Alchemy webhook	Alchemy webhook received	None — wait for block confirmations
ALCHEMY_VERIFIED	Confirmed on-chain via Alchemy API	Block threshold reached	System auto-advances — no action
ON_CHAIN_CONFIRMED	Transaction fully confirmed on blockchain	Confirmation threshold met	ASIG reconciles — no action needed
RECONCILED	Settlement complete — funds reconciled ✓	Admin reconciliation	Process complete
MANUAL_REVIEW	Flagged: RPC error / TRON / amount mismatch	Verification anomaly	Contact ASIG ops with payload_id
FAILED	Blockchain verification failed or mismatch ✗	Verification failed	Resubmit with corrections or contact ops

6.2 OutboundTransferStatus — ASIG Sending Payments

Status	Meaning	Notes
PENDING	Transfer created — awaiting processing	Initial state after creation
AWAITING_APPROVAL	Requires admin approval before broadcast	Large transfers trigger approval workflow
APPROVED	Admin approved — queued for broadcast	Pre-broadcast holding state
BROADCASTING	Transaction being submitted to network	Broadcast in progress
COMPLETED	Confirmed on-chain — settlement complete ✓	Final success state
FAILED	Broadcast failed — retry or manual fix ✗	Check error details in transfer record
CANCELLED	Cancelled before broadcast	No funds moved

6.3 OrderStatus — Payment Orders (Coinbase / MoonPay / Circle / Stripe)

Status	Meaning
CREATED	Order record created — awaiting provider confirmation
PENDING	Submitted to payment provider — payment pending
PROCESSING	Provider is actively processing the payment
COMPLETED	Payment confirmed and settlement complete ✓
FAILED	Payment failed — contact provider for details ✗
REFUNDED	Payment was refunded to the customer
EXPIRED	Order expired before payment was received

7. ALL ERROR CODES & REJECTION REASONS

All ASIG error responses use a consistent JSON body. The error_code field uniquely identifies the rejection reason. Every rejection is also written to the audit log with the event_type shown below.

Error response structure:

```
{ "error": "error_code_string",
"message": "Human-readable description of the rejection" }
```

HTTP	error_code	Audit Event	Cause	Corrective Action
401	missing_credentials	PAYLOAD_REJECTED_AUTH	No X-API-Key and no Authorization header	Add X-API-Key header or OAuth Bearer token
401	invalid_api_key	PAYLOAD_REJECTED_AUTH	API key not found or marked inactive	Verify key in ALSHUMOOKH admin dashboard
401	oauth_required	PAYLOAD_REJECTED_AUTH	Client requires OAuth2 but API Key was sent	Obtain Bearer token via /api/v1/oauth/token
403	ip_not_allowed	PAYLOAD_REJECTED_AUTH	Caller IP not in the client IP allowlist	Register your egress IP in ASIG dashboard
400	missing_idempotency_key	PAYLOAD_REJECTED_VALIDATION	Idempotency-Key header is absent	Add Idempotency-Key: <uuid4> to every request
401	mtls_required	PAYLOAD_REJECTED_AUTH	Client requires mTLS but no cert fingerprint	Add X-Client-Cert-Fingerprint header
400	invalid_jwe	PAYLOAD_REJECTED_VALIDATION	JWE-encrypted body failed decryption	Check JWE key pair and algorithm settings
401	timestamp_expired	PAYLOAD_REJECTED_AUTH	X-Timestamp is older than 5 minutes	Sync server clock to UTC via NTP
401	invalid_jws	PAYLOAD_REJECTED_AUTH	JWS signature on body did not verify	Check signing key and JWS algorithm
401	hmac_required	PAYLOAD_REJECTED_AUTH	Client requires HMAC but none provided	Add X-HMAC-Signature + X-Timestamp headers
401	invalid_hmac	PAYLOAD_REJECTED_AUTH	HMAC-SHA256 signature verification failed	Check secret key and message construction
400	invalid_json	—	Request body is not valid JSON	Set Content-Type: application/json
400	invalid_payload	—	Body is not a JSON object (e.g. array or string)	Body must be a JSON object { }
404	not_found	—	GET payload/{id} — payload_id does not exist	Use payload_id returned from /ingest response
429	rate_limit_exceeded	—	Exceeded 200/60s per key or 240/60s global	Implement exponential backoff and request queue

8. COMPLETE API ENDPOINTS REFERENCE

8.1 Settlement & Payload Endpoints

Method	Endpoint	Auth	Description
POST	/api/v1/payloads/ingest	API Key + Idempotency	Submit inbound settlement payload — main integration endpoint
GET	/api/v1/payloads/schema	API Key	Retrieve canonical schema and supported field aliases
POST	/api/v1/payloads/m1-fund	API Key	M1 Fund inbound settlement (EUR wire / IBAN pathway)
GET	/api/v1/payloads/{id}	API Key	Get payload status and full details by payload_id
POST	/api/v1/oauth/token	Client credentials	Obtain OAuth2 Bearer token (expires in 3600 seconds)

8.2 Payment Order Endpoints

Method	Endpoint	Auth	Description
POST	/api/v1/payments/orders	API Key	Create payment order (Coinbase / MoonPay / Circle / Stripe)
GET	/api/v1/payments/orders	API Key	List all payment orders with pagination
GET	/api/v1/payments/orders/{id}	API Key	Get payment order by ID
GET	/api/v1/payments/orders/{id}/status	API Key	Get real-time order status
GET	/api/v1/payments/widget-url	API Key	Get onramp widget URL
GET	/api/v1/payments/coinbase/onramp-url	API Key	Get Coinbase onramp URL
GET	/api/v1/payments/transactions	API Key	List all transactions
GET	/api/v1/payments/transactions/{id}	API Key	Get transaction detail by ID

8.3 Webhook Receiver Endpoints (Blockchain & Payment Events)

Method	Endpoint	Auth	Description
POST	/webhooks/alchemy	Alchemy token	Blockchain confirmations — auto-advances payload verification status
POST	/webhooks/circle	Circle HMAC	Circle payment confirmation events
POST	/webhooks/moonpay	MoonPay signature	MoonPay payment status events
POST	/webhooks/coinbase	Coinbase signature	Coinbase Commerce payment events

8.4 Admin Endpoints (ASIG Operations / Authorized Partners Only)

Method	Endpoint	Auth	Description
GET	/api/v1/admin/payloads	Admin key	List all payloads with filters: status, date, network, client
GET	/api/v1/admin/payloads/{id}	Admin key	Full payload detail with complete audit trail
POST	/api/v1/admin/payloads/{id}/verify	Admin key	Trigger manual blockchain re-verification
PATCH	/api/v1/admin/payloads/{id}/review	Admin key	Update manual review decision
POST	/api/v1/admin/outbound-transfers	Admin key	Initiate USDT outbound transfer (ETH / TRON / Base)
GET	/api/v1/admin/outbound-transfers	Admin key	List outbound transfers with full status history

Method	Endpoint	Auth	Description
POST	`/api/v1/admin/clients`	Admin key	Create new API client record
GET	`/api/v1/admin/clients`	Admin key	List all registered API clients
GET	`/api/v1/admin/dashboard/stats`	Admin key	System statistics and payload counts by status
GET	`/api/v1/admin/treasury/balance`	Admin key	Treasury wallet balances across all networks
POST	`/api/v1/admin/fnfcu`	Admin key	FNFCU transfer request submission

9. BLOCKCHAIN NETWORKS, CONTRACTS & RECEIVER ADDRESSES

9.1 Supported Token Contracts

Token	Network	Contract Address	Standard	Status
USDT	Ethereum Mainnet	`0xdAC17F958D2ee523a2206206994597C13D831ec7`	ERC-20	**Active**
USDT	Tron	`TR7NHqjeKQxGTCi8q8ZY4pL8otSzgjLj6t`	TRC-20	**Placeholder**
USDC	Ethereum Mainnet	`0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48`	ERC-20	**Active**
USDC	Base Network	`0x833589fcd6edb6e08f4c7c32d4f71b54bda02913`	ERC-20	**Active**
M1 Token	Sepolia Testnet	`0xD999DB972BBDc2C13a9595A1474A04F5e59169a5`	ERC-20	Testnet
SIG Token	Sepolia Testnet	`0x14E068D3C99cbC8BAe723D83D1761EF4C124A0ab`	ERC-20	Testnet

9.2 ASIG Receiver Wallets & Bank Details

Channel	Address / Account Number	Asset Accepted
Ethereum / Base (EVM)	`0xBD682cfD8382a90adfDd6745780D3D7959c4d939`	USDT ERC-20 · USDC
Tron (TRC-20)	Contact ASIG operations for Tron address	USDT TRC-20
Wire Transfer IBAN	`AE960260001015754367301`	USD / EUR (fiat)
Bank Name	Emirates NBD — Sheikh Zayed Road, Dubai, UAE	—
BIC / SWIFT Code	`EBILAEADXXX`	—

9.3 Blockchain Confirmation Thresholds

Network	Chain ID	Min Confirmations	Avg Time	Verification Method
Ethereum Mainnet	1	6 blocks	~90 seconds	Alchemy Webhook + EVM RPC
Base Network	8453	6 blocks	~18 seconds	Alchemy Webhook + EVM RPC
Tron	N/A	20 blocks	~60 seconds	MANUAL_REVIEW (placeholder)
Sepolia Testnet	11155111	6 blocks	~90 seconds	Alchemy Webhook (testing only)

10. WEBHOOK & CALLBACK SPECIFICATION

When a payload's verification_status changes, ASIG sends a JSON POST to the callback_url provided in the original payload. The endpoint must respond HTTP 200 within 10 seconds. ASIG retries 3 times with exponential backoff on failure.

10.1 Outbound Callback Payload (ASIG → HIPERCAPITAL)

```
POST https://your-server.com/webhook/asig
Content-Type: application/json
X-ASIG-Signature: <hmac_sha256_hex>

{
"event": "payload_status_update",
"payload_id": "a8f2c3d4-1234-5678-abcd-ef0123456789",
"transaction_reference": "HIPC-2026-001",
"verification_status": "ON_CHAIN_CONFIRMED",
"previous_status": "ALCHEMY_VERIFIED",
"tx_hash": "0xabc123...def456",
"network": "ethereum",
"amount": "50000.00",
"asset": "USDT",
"timestamp": "2026-06-09T10:05:22Z"
}
```

10.2 Callback Signature Verification

```
# Verify the ASIG callback is authentic:
import hmac, hashlib

received = request.headers['X-ASIG-Signature']
expected = hmac.new(
your_webhook_secret.encode(),
request.body,
hashlib.sha256
).hexdigest()

if not hmac.compare_digest(received, expected):
raise Exception('Invalid signature — reject request')
```

11. M1 FUND / WIRE TRANSFER PATHWAY

For EUR-denominated or fiat wire settlements, use the M1 Fund inbound pathway. ASIG converts incoming EUR wire transfers to USDT via FX conversion, then mints M1 Tokens (ERC-20 on Sepolia testnet).

Field	Value
IBAN	`AE960260001015754367301`
BIC / SWIFT Code	`EBILAEADXXX`
Bank Name	Emirates NBD — Sheikh Zayed Road, Dubai, UAE
M1 Ingest Endpoint	`POST /api/v1/payloads/m1-fund`
M1 Receiver Wallet (ETH)	`0xBD682cfD8382a90adfDd6745780D3D7959c4d939`
M1 Token Contract (Sepolia)	`0xD999DB972BBDc2C13a9595A1474A04F5e59169a5`
Tokenization Flow	EUR Wire → FX Conversion → USDT → M1 Token (ERC-20 on Sepolia)
settlement_type Value	M1_FUND_INBOUND (auto-set by the /m1-fund endpoint)

11.1 M1 Fund Request Example

```
POST /api/v1/payloads/m1-fund
X-API-Key: asig_live_xxxx
Content-Type: application/json

{
"grouped_m1_detail": {
"total_group_value_eur": "250000.00",
"group_reference": "HIPC-M1-2026-001",
"settlement_date": "2026-06-09"
},
"sender": "HIPERCAPITAL FINANCE S.L.",
"receiver": "ALSHUMOOKH GROUP",
"callback_url": "https://your-server.com/webhook/asig"
}
```

12. INTEGRATION PHASES & COMPLIANCE REQUIREMENTS

PHASE 1 — Legal & Regulatory (Weeks 1 – 4)

Requirement	Detail
FCA Authorization	Obtain FCA authorization for UK payment activities, or appoint an FCA-authorized partner
Reserve Audit	Third-party audit of the 6.98 trillion EUR reserve claim from a Big 4 firm or equivalent
AML/KYC Docs	Provide AML policy and KYC program documentation — ASIG requires FATF-compliant counterparties
Corporate Verification	UK Companies House certificate + Spain CNMV registration + Panama incorporation docs

PHASE 2 — Technical Setup (Weeks 5 – 8)

Requirement	Detail
USDT Bridge	Deploy HCEURO → USDT bridge on Ethereum or Base — HCEURO has no external liquidity today
API Client Setup	Contact ASIG operations to create client record and receive: API key, HMAC secret, client ID
IP Registration	Provide production egress IP range for registration in ASIG firewall allowlist
Webhook Endpoint	Deploy public HTTPS endpoint to receive ASIG status callbacks (see Section 10)
HMAC Integration	Implement HMAC-SHA256 request signing per Section 3.3 specification

PHASE 3 — Sepolia Testnet Integration (Weeks 9 – 12)

Requirement	Detail
Testnet Credentials	Request Sepolia testnet API key — all tests run against Chain ID 11155111
10 Test Payloads	Submit 10 test payloads with real Sepolia USDT tx_hash values
Status Flow Test	Confirm progression: ALCHEMY_PENDING → ALCHEMY_VERIFIED → ON_CHAIN_CONFIRMED
Callback Test	Confirm webhook delivery to your callback_url for every status transition
Error Handling Test	Intentionally trigger each error_code to verify your client error handling

PHASE 4 — Production Go-Live (Weeks 13 – 16)

Requirement	Detail
Security Review	ASIG security team reviews your integration architecture and code
Load Test	Demonstrate sustained 200 req/60s without triggering rate limits
First Live Payment	Submit first production USDT payment with real on-chain tx_hash
Reconciliation	ASIG operations confirms first RECONCILED status payload
SLA Agreement	Sign ASIG integration SLA — 99.9% uptime guarantee, 48h support SLA

13. LIVE cURL REQUEST EXAMPLES

13.1 Get OAuth2 Bearer Token

```
curl -X POST https://api.alshumookh.com/api/v1/oauth/token \
-H 'Content-Type: application/x-www-form-urlencoded' \
-d 'grant_type=client_credentials
&client_id=YOUR_CLIENT_ID
&client_secret=YOUR_CLIENT_SECRET'
```

13.2 Submit Settlement Payload with API Key + HMAC

```
curl -X POST https://api.alshumookh.com/api/v1/payloads/ingest \
-H 'Content-Type: application/json' \
-H 'X-API-Key: asig_live_xxxxxxxxxxxxxxxxxxxx' \
-H 'Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000' \
-H 'X-HMAC-Signature: 3a7f9c2e4b8d1f...' \
-H 'X-Timestamp: 1749461234' \
-d '{
"transaction_reference": "HIPC-2026-001",
"tx_hash": "0xabc123...def456",
"sender_wallet": "0xYOUR_SOURCE_WALLET",
"receiver_wallet": "0xBD682cfD8382a90adfDd6745780D3D7959c4d939",
"amount": "50000.00",
"asset": "USDT",
"network": "ethereum",
"token_contract": "0xdAC17F958D2ee523a2206206994597C13D831ec7",
"callback_url": "https://your-server.com/webhook/asig"
}'
```

13.3 Submit Using OAuth2 Bearer Token

```
curl -X POST https://api.alshumookh.com/api/v1/payloads/ingest \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...' \
-H 'Idempotency-Key: 661f9511-f3ab-52e5-b827-557766551111' \
-H 'Content-Type: application/json' \
-d '{"txHash":"0xabc...","fromAddress":"0xYOUR",
"toAddress":"0xBD682c...","tokenAmount":"50000",
"tokenSymbol":"USDT","chainName":"ethereum"}'
```

13.4 Check Payload Status

```
curl https://api.alshumookh.com/api/v1/payloads/a8f2c3d4-1234-5678-abcd-ef0123456789 \
-H 'X-API-Key: asig_live_xxxxxxxxxxxxxxxxxxxx'
```

13.5 Get Schema Definition

```
curl https://api.alshumookh.com/api/v1/payloads/schema \
-H 'X-API-Key: asig_live_xxxxxxxxxxxxxxxxxxxx'
```

14. GO-LIVE COMPLIANCE CHECKLIST

ALL items must be completed and verified by ALSHUMOOKH GROUP before HIPERCAPITAL FINANCE receives production API credentials. Items marked BLOCKER prevent go-live regardless of progress on other items.

#	Category	Requirement	Evidence Required	Priority
1	**Legal**	FCA Authorization or equivalent financial license	License certificate + regulator register entry	**BLOCKER**
2	**Legal**	AML / KYC program documentation	AML policy document + compliance officer contact	**BLOCKER**
3	**Legal**	Corporate structure + beneficial ownership declaration	Entity chart + UBO declaration	**Required**
4	**Financial**	Independent reserve audit — 6.98 trillion EUR claim	Big 4 or equivalent audit report	**BLOCKER**
5	**Financial**	HCEURO → USDT bridge deployed on public chain	Bridge contract address + audit + liquidity proof	**BLOCKER**
6	**Technical**	10 successful Sepolia test payloads	ASIG payload IDs with RECONCILED status	**Required**
7	**Technical**	HMAC-SHA256 signing verified	Demo request with valid X-HMAC-Signature header	**Required**
8	**Technical**	Webhook endpoint deployed and tested	All status transitions received and acknowledged	**Required**
9	**Technical**	Idempotency-Key implementation verified	Duplicate request correctly returns replay response	**Required**
10	**Technical**	Rate limit compliance demonstrated	Load test results — zero rate limit violations	**Required**
11	**Operations**	Production IP addresses registered in ASIG	Egress IP list provided to ASIG operations team	**Required**
12	**Operations**	Integration SLA agreement signed	Signed SLA document returned to ALSHUMOOKH GROUP	**Required**

■ 4 BLOCKERS MUST BE RESOLVED FIRST:

Items 1, 2, 4, and 5 are BLOCKERS. Production API access cannot be granted until all four are resolved.

The USDT bridge (item 5) is the primary technical blocker — HCEURO has no external market liquidity and ASIG cannot accept private ledger assets under any circumstances.